



We notice that the output has another representation — **HEX**.

This helps us understand that the value starting with **3030...** is **hexadecimal**.

When we convert it to text, we obtain a **binary string**:

```
00111000011010010111010101100111001100000111010001110001010100110100100000111000
```

In the image, we can see a **Siberian Husky** dog, which hints that the next step is related to **binary/ASCII conversion**.

We convert the binary string to ASCII using an online converter, which results in:

```
8iug0tqSH8
```

Barcode Analysis

At the bottom of the image, there is a barcode containing the numbers:

```
68775656
```

We assume this value is also **hexadecimal**, so we convert it to ASCII.

The result is:

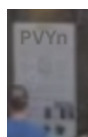
```
hwVV
```

Next, we apply a **Caesar (Shift) Cipher with a -3 shift**, which gives:

```
etSS
```

Final Clue

Finally, by zooming in on the image (hinted by the computer being set to a zoomed display), we find **four letters** hidden in the Roman building:



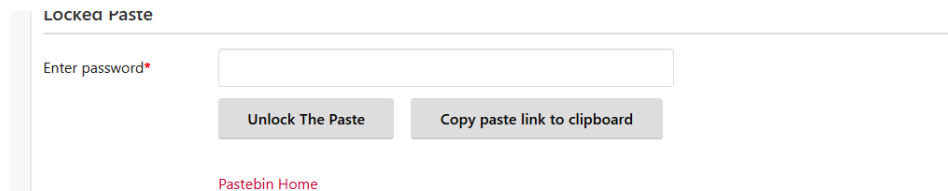
We convert it using a **Caesar (Shift) Cipher with a -3 shift** and obtain:

MSVk

Next, we combine the two 4-character strings and append them to the URL:

<https://pastebin.com/MSVketSS>

This successfully leads us to the next stage.



we are asked to enter a password - we try the **8iug0tqSH8**

we get in!!

Amazing!

1. You passed the 1st Stage
2. So far we were warming up....
- 3.
4. 2nd Stage: Read it carefully & pay attention to everything....
5. -----
6. why do I even bother writing rules...
- 7.
8. same pattern again.
9. told myself not to use it.
- 10.
11. it always starts the same.
12. a habit I never really broke.
- 13.
14. letters first.
15. numbers take over.
- 16.
17. they keep growing.
18. one after another.
- 19.
20. hate when systems force special chars.
21. still easier to just put it at the end.
- 22.
23. this is why breaches happen.
- 24.
25. z0rr0s

- 26. -----
- 27.
- 28. # Download the following link and continue with Stage 2
- 29. <https://drive.proton.me/urls/P2CTHE8WWG#a7E6wPObfg1H>

Part 2 – OSINT

We analyzed the files provided in the downloaded folder and realized that the challenge revolves around a **Git repository**.

We attempted to search online for unique strings from the files using Google and Google Docs, but without success.

After further analysis, we discovered that it is possible to locate Git repositories using **commit messages** alone.

Inside the `commit_notes.txt` file, we found the following commit message:

“Temporary archive added for testing”

Based on this insight, we downloaded the **GitHub CLI** in order to search for a repository using this commit message — a very powerful OSINT technique.

```
gh search commits "Temporary archive added for testing"
```

```
C:\Users\The user>gh search commits "Temporary archive added for testing"
Showing 1 of 1 commits
REPO          SHA          MESSAGE          AUTHOR    CREATED
z0rr0s/legacy-assets  e3354ceb58283fcb9bdfd190e138d36f... Temporary archive added for testing z0rr0s  about 4 days ago
C:\Users\The user>
```

we search the url : <https://github.com/XXXXXXXXXX>

and we download the zip file there **assets_2018.rar**.

Part 3 (Steganography)

we find in the page on the repository this folder

<https://github.com/z0rr0s/legacy-assets/releases/tag/legacy-2018-assets>

we download the folder and unzip it:

logs	2/10/2026 4:54 PM	File folder
misc	2/10/2026 4:51 PM	File folder
media	2/10/2026 4:50 PM	File folder
notes	2/10/2026 1:20 PM	File folder
images	2/10/2026 2:12 AM	File folder

we read through the files

in the **mist** folder we find this **old_hosts.conf** file with a old IP address

legacy hosts (partial)
staging-old = **129.158.220.16**
test-lab = 10.10.20.55
prod removed after audit

now reading the **ssh_keys_inventory.csv** excel file

	E	D	C	B	A
	notes	status	owner	type	key_name
	prefix only	active	feature	ed25519	id_ed2551
	suffix only	active	feature	tag	oracle_fea
	weak key - revoked		legacy-adr	rsa	id_rsa_leg
	unused aft expired		ops	rsa	id_rsa_bac

we see that the owner is **feature**

reading the **key_rotation_ticket.md** in the **misc** folder file we see this line
- Feature access remains legacy-only

now we guess the username is **Feature!**

reading the **audit_chat.log** file we understand that the ip we found above is not the axact adress

[2018-09-12 10:14] amy: audit feedback just came in. stop putting hosts in tickets.
[2018-09-12 10:15] dan: yeah, they flagged that twice.
[2018-09-12 10:16] dan: keep feature access isolated. do NOT reuse default or prod.
[2018-09-12 10:17] amy: agreed. feature only. same scope as freeze.
[2018-09-12 10:18] amy: and don't attach the full address here.
[2018-09-12 10:19] dan: ok. last octet only if needed.
[2018-09-12 10:20] dan: freeze note says two-four-nine.
[2018-09-12 10:21] amy: old staging was sixteen. that box is dead.
[2018-09-12 10:22] dan: good. if anyone asks for the rest, tell them to sing it.
[2018-09-12 10:23] amy: lol. auditors would love that.
[INFO] feature host rotated after incident #249

we save the number **249**

now listening to the audio file **לא על הראשי.wav** in the media folder- the song is all about a **key to ssh** - we understand that we need to look for a **OPENSSH PRIVATE KEY** not a password.

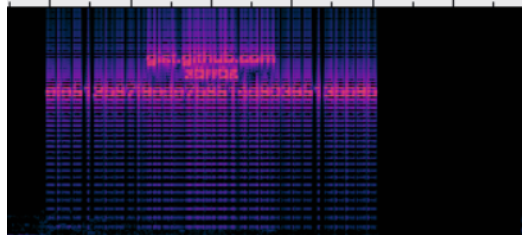
listanig to the **כמעט הכתובת.wav** audio we hear there that the number 16 is old and 249 is the last without a dot - so we discover that the new ip is **129.158.220.249** not **129.158.220.16**

in the end of the song its saids to look or something in the **waves** not in the words.

we use <https://www.audacityteam.org>
to see the waves of the song, we use:

Spectrogram Settings

and yes we find in the beginning a readable txt.



when reading closely we see the address <https://gist.github.com>
and user **z0rr0s**

we find the link <https://gist.github.com/z0rr0s/efe512b87f9ede7b8514d8036513bb9b>
and there we see the OPENSSH PRIVATE KEY

```
id_ed25519_oracle_feature
1  -----BEGIN OPENSSH PRIVATE KEY-----
2  b3B1bnNzaC1rZXktdjEAAAABG5vbmUAAAABm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
3  QyNTUxOQAAACAhmCP9NbxnAU5xRfTzIY83C3kaP1eMoe2MEzZzkaxV5gAAAJBhNCG1YTQh
4  pQAAAAtzc2gtZWQyNTUxOQAAACAhmCP9NbxnAU5xRfTzIY83C3kaP1eMoe2MEzZzkaxV5g
5  AAAEB2408MjPrE3DyeNN2xSwzMuE3g0Jdf2D6Zkwvs+4ZYySGYI/01vGcBTnFF9PMhjzcL
6  eRo+V4yh7YwTNnORrFXmAAAAC3VidW50dUBjdGYxAQI=
7  -----END OPENSSH PRIVATE KEY-----
```

now we copy the key to a file named **sshkey** and try to connect with all the info we collected

```
ssh -i sshkey feature@129.158.220.249
```

we get connexion

```

$ ssh -i sshkey feature@129.158.220.249
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-1081-oracle x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Feb 18 14:20:23 UTC 2026

System load:  0.0               Processes:    126
Usage of /:   12.0% of 44.96GB   Users logged in:  0
Memory usage: 35%              IPv4 address for ens3: 10.0.0.248
Swap usage:   0%

 * Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
   on 31 May 2025.

   For more details see:
   https://ubuntu.com/20-04

Expanded Security Maintenance for Infrastructure is not enabled.

8 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

93 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04

Last login: Tue Feb 17 17:25:03 2026 from 79.177.143.212
feature@ctf1:~$ ls
ctf_stage4
feature@ctf1:~$ cd ctf_stage4/
feature@ctf1:~/ctf_stage4$ ls
FinalBoss.rar  Final_Note.txt

```

Part 4 (Brute-Force)

In the **ctf_stage4** folder we find **FinalBoss.rar** folder and a **Final_Notes.txt** file.

we try to unrar the **FinalBoss.rar** but it Requires a passwords 😞

we open the note to find are mask for the brute force

You already know this habit.
 It always begins loud,
 then quickly calms down.
 After that?
 I stop counting.
 Digits just pile up.
 One symbol.
 At the end.
 To satisfy the policy.
 Same structure.
 Every time.
 The server was never meant for cracking.
 Move the artifact.
 Use the right environment.

first we get the hash of the rar folder using rar2john

```
rar2john FinalBoss.rar > hash.hashcat
```

to clean the hash file and to make sure its the right hash we run this

```
rar2john FinalBoss.rar | head -n 1 | awk -F ":" '{print $2}' > finalboss.hashcat
```

this is the **hash** of the folder

```
$rar5$16$f4ef688cd97384f4f7061fa0d1a27d67$15$10023ca8da240416c44728a232e7a30e$8$ee52d1
```

Finding the Mask Attack

After reviewing the note we found, we can infer the password structure:

- It starts with **one uppercase letter** — *"It always begins loud"*
- Followed by **one lowercase letter** — *"then quickly calms down"*
- Then **digits** — *"Digits just pile up"* (the exact number is not immediately clear)
- Ends with **one symbol** — *"One symbol at the end"*

The note also mentions *"Same structure every time"*, which suggests the password always has the **same length and format**.

From the Pastebin file with similar clues, we noticed the line:

"This is why breaches happen" — indicating that an older password was **8 characters long**, which led to the breach.

Given the structure:

- 1 uppercase letter
- 1 lowercase letter
- 5 digits
- 1 special character

We conclude the password length is **8 characters**, with **5 digits** in the middle.

Running the Attack on a GPU Machine

We deployed a GPU instance using RunPod:

```
https://console.runpod.io/deploy?gpu=RTX+PR0+6000&count=1&template=runpod-torch-v280
```

After creating an SSH key and connecting to the machine, we ran the following commands in order:

```
wget https://hashcat.net/files/hashcat-6.2.6.7zsudo apt updatesudo apt install 7zip
7z x hashcat-6.2.6.7zcd hashcat
nano hash.hash
```

(Here we pasted the full output from `rar2john`.)

```
cut -d: -f2- hash.hash > finalboss.hashcat
```

Finally, we executed the mask attack:

```
./hashcat.bin -m 13000 -a 3 finalboss.hashcat ?u?l?d?d?d?d?s -0 -w 4 --status --status-time  
r 10
```

we get the password for the cheat sheet

\$rar5\$16\$f4ef688cd97384f4f7061fa0d1a27d67\$15\$10023ca8da240416c44728a232e7a30e\$8\$ee52d160d865c49e:XXXXX

DONE!!!